

# Radical Fault Tolerance

Roots of processes, arbitration, and the resolver that consensus supplies

L. G. Meredith

F1R3FLY.io Research Note  
August 2026

## Abstract

The square root of a process  $P$  is a process  $\sqrt{P}$  such that  $\sqrt{P} \mid \sqrt{P}$  is bisimilar to  $P$ ; the  $n$ -th root is the corresponding  $n$ -fold statement. This note works out, in detail, a construction sketched in a 2018 RChain research deck. Three things are established and one is proposed. First, exact roots are governed by divisibility: modulo a unique-parallel-decomposition hypothesis, a process has an  $n$ -th root exactly when every parallel prime occurs with multiplicity divisible by  $n$ , so almost no process has one, while idempotent processes have all of them trivially. Second, the naive root computed by structural recursion is correct on the determinate fragment but fails at contention, and it fails for two independent reasons: multiplicity of messages is observable to arbitrary contexts, which forces the correctness statement to be relative to the image of the root map; and the copies of a doubled race may resolve it differently, which no rearrangement of the clauses can repair. Third, the repair is an *arbiter*: a term that joins on the contended deposits, contributes a bit drawn from its own copy's non-determinism, and decides by a function symmetric in the contributions, so that every copy decides alike. Symmetry is the agreement condition, and it is what the original sketch's decision table silently supplied. Finally, we argue that because contention is a runtime rather than a syntactic property, the arbiter cannot be installed per race but must own the channel, so that the fixed point of the construction is a node that serialises a namespace — which is the sense in which taking roots and running consensus are the same activity. A further section separates three things the word *asynchrony* is made to carry — asynchronous sending, unbounded delay, and unreliable delivery — and reifies the medium as a process, which puts the fault model inside the calculus as a perturbation of a term rather than beside it as an assumption, and which exhibits the arbiter as a channel that  $n$  copies are constrained to agree on. Throughout, the arbiter is read as an instrument for making a family of independent *resolvers* of nondeterminism behave as one. A rholang implementation of the square root of a racing process accompanies the note.

## 1 Introduction

Fault tolerance by replication starts from a program and makes copies of it. The copies are not independent: they are arranged so that  $n$  executions amount to one execution. Written as an equation in a process calculus, that arrangement says something arithmetic. If  $R$  is what each of  $n$  nodes runs, and the network of nodes is to be indistinguishable from a single execution of  $P$ , then

$$R \parallel^n = \underbrace{R \mid \cdots \mid R}_n \approx P,$$

which is to say that  $R$  is an  $n$ -th root of  $P$  under parallel composition. The pun in the title is the observation: a replicated system is a radical, and a consensus protocol is whatever makes the radical exist.

The equation is due to a 2018 RChain research deck, which stated it, gave a structural recursion computing roots, exhibited the obstruction that recursion meets at a race, sketched a coordinating process  $C$  that removes the obstruction for a single two-message race, noted that  $C$  assumes reliable delivery and honesty — “this is effectively the difference between  $C$  and Casper” — and left the  $n$ -ary case as an exercise. This note works the sketch out.

**What we found.** Four things in the sketch needed correcting or supplying, and each turned out to be load-bearing rather than clerical.

1. The decision table in the sketch is symmetric in its two arguments — the two agreeing pairs choose one payload, the two disagreeing pairs choose the other — and that symmetry is exactly what makes the copies agree, since they read the contributions in whatever order they happen to. Symmetry is the agreement condition (Lemma 6.5), and it also makes the shared bit unbiased when either copy’s local coin is (Lemma 6.6).
2. The multi-binding receipts must be *joins*. In the notation of rholang 1.4,  $\&$  joins within a group and  $;$  separates groups; read sequentially, the two arbiters can each take one half of what they need and deadlock (Lemma 6.3).
3. The arbiter consumes both contending messages and re-emits only the winner, so the loser is destroyed — but the source process retains it, and retains its barb. The loser has to be parked, and it cannot be parked on the delivered channel without re-opening the very divergence the arbiter was installed to close (Remark 6.8).
4. Multiplicity of messages is observable, so  $\sqrt{P} \mid \sqrt{P} \approx P$  is false against arbitrary contexts even for  $P = x!(0)$ , where no race occurs at all (Proposition 4.4). The correctness statement has to be relative to a class of contexts, and the right class is the image of the root map: the observer is replicated along with the observed.

**What is new here.** Section 3 places exact roots in the arithmetic of the process monoid, where they are governed by divisibility and are therefore rare. Section 7 argues that contention is a runtime property, not a syntactic one, so an arbiter cannot be attached to a race the way the sketch attaches it; it must own the channel, and the general contention site requires agreement on a matching rather than on a bit. Section 8 reads the whole construction in terms of *resolvers*: the rho calculus is silent about how nondeterminism is fulfilled, and the root equation is the first equation we know of in which that silence is not benign, since it demands that  $n$  independently resolved copies behave as one resolved copy.

**What is not claimed.** The relationship to consensus is presented as an analogy (Section 10) and not as a theorem. We say what corresponds to what, and say plainly which of the standard correctness conditions our lemmas are shadows of, but we do not define a fault model and prove an impossibility result inside it. Threshold roots — the  $k$ -of- $n$  weakening that would make the construction fault *tolerant* rather than merely fault *replicating* — are named in Section 13 and not developed.

## 2 The calculus, and what we observe

We work in the rho calculus [11]. Names are quoted processes; there is no name restriction operator, and freshness is obtained by construction rather than by binding.

$$\begin{aligned} P, Q &::= 0 \mid \text{for}(y \leftarrow x)P \mid x!(P) \mid *x \mid P \mid Q \\ x, y &::= @P \end{aligned}$$

Structural congruence  $\equiv$  makes  $(\cdot \mid \cdot, 0)$  a commutative monoid and identifies  $*@P$  with  $P$ . The single reduction rule is communication:

$$\text{for}(y \leftarrow x)P \mid x!(Q) \longrightarrow P\{@Q/y\}.$$

Receipts are linear by default;  $\text{for}(y \leftarrow x)P$  denotes the persistent receipt, which survives the rendezvous. Substitution does not descend under a quote. We write  $\text{for}(y_1 \leftarrow x_1 \ \& \ \cdots \ \& \ y_m \leftarrow x_m)P$  for the  $m$ -fold join, whose firing is atomic: it consumes one datum at each of the  $m$  listed channels in a single step, or it does not fire.

**Remark 2.1.** The join arity matters throughout this note, and joins on a *repeated* channel are used essentially:  $\text{for}(e_1 \leftarrow o \ \& \ e_2 \leftarrow o)P$  consumes two data from  $o$  at once, and the two data may be assigned to the two patterns in either order. Every arbitration below turns on that atomicity and on that order-indifference.

**Definition 2.2** (Observation).  $P \downarrow_x$  iff  $P \equiv x!(Q) \mid R$  for some  $Q, R$ ;  $P \Downarrow_x$  iff  $P \longrightarrow^* P'$  for some  $P' \downarrow_x$ . Weak barbed bisimilarity, written  $\approx_\bullet$ , is the largest symmetric relation closed under reduction and preserving weak barbs. For a class  $\mathcal{D}$  of contexts,  $P \approx_{\mathcal{D}} Q$  iff  $C[P] \approx_\bullet C[Q]$  for every  $C \in \mathcal{D}$ ; taking  $\mathcal{D}$  to be all rho calculus contexts gives barbed congruence, written  $\approx$ .

Weak barbed bisimilarity is insensitive to the multiplicity of a message:  $x!(0) \approx_\bullet x!(0) \mid x!(0)$ , since both have the barb at  $x$  and neither reduces. Barbed congruence is not, and Proposition 4.4 turns that difference into the first design constraint on the root map.

## 3 Roots and divisibility

**Definition 3.1** (Root). Let  $\simeq$  be an equivalence on processes that is a congruence for  $\mid$ . A process  $R$  is an  $n$ -th root of  $P$  with respect to  $\simeq$  iff  $R \parallel^n \simeq P$ .

Roots are not unique — Definition 3.1 defines a relation — so  $\sqrt[n]{P}$  always denotes a chosen *construction* of a root and never “the” root. The first question is how many processes have one at all, and the answer is arithmetic. Write  $\mathcal{M} = (\mathcal{P}/\simeq, \mid, 0)$ , a commutative monoid. In additive notation an  $n$ -th root is a solution of  $nX = P$ , so the question is one of divisibility in  $\mathcal{M}$ .

**Definition 3.2.**  $P$  is *parallel prime* iff  $P \not\simeq 0$  and  $P \simeq Q \mid R$  implies  $Q \simeq 0$  or  $R \simeq 0$ .

**Hypothesis 3.3** (Unique decomposition). On the fragment under consideration, every process is  $\simeq$  a finite parallel product of parallel primes, uniquely up to permutation of the factors.

Hypothesis 3.3 is a genuine hypothesis and not a citation. Unique parallel decomposition is a theorem for finite behaviours in CCS-like calculi modulo strong bisimilarity [13], and has been substantially generalised [10]; the weak case is delicate, and neither the reflective structure of the rho calculus nor infinite behaviour is covered by the existing results. We use it to say what the arithmetic *would* be, and we flag in Section 13 that establishing or refuting it for the rho calculus is the first piece of unfinished business.

**Proposition 3.4** (Divisibility). *Assume Hypothesis 3.3 and that  $\mathcal{M}$  has no idempotents other than 0. Then  $\mathcal{M}$  is free commutative on the parallel primes, and  $P$  has an  $n$ -th root iff  $n$  divides the multiplicity of every prime factor of  $P$ . When a root exists it is unique.*

*Proof.* A free commutative monoid on a set  $S$  is  $\bigoplus_S \mathbb{N}$ , in which  $nX = P$  has a solution iff every coordinate of  $P$  is divisible by  $n$ , and then exactly one.  $\square$

**Proposition 3.5** (Idempotents). *If  $P \mid P \simeq P$  then  $P$  is an  $n$ -th root of itself for every  $n$ .*

The rho calculus instance of Proposition 3.5 is the persistent receipt: a pure server for  $(y \leftarrow x)Q$ , with  $Q$  not itself contending, satisfies  $\text{for}(y \leftarrow x)Q \mid \text{for}(y \leftarrow x)Q \approx_\bullet \text{for}(y \leftarrow x)Q$ , because each copy survives every rendezvous and the pair delivers what one delivers. So the interesting content of the root equation is a statement about *linear* receipts, which is also why the sketch's obstruction is where it is: a linear receipt makes the loser of a race into a durable record of which branch was taken.

**Proposition 3.6.**  *$x!(0)$  is parallel prime for barbed congruence, and hence has no  $n$ -th root for  $n \geq 2$ .*

*Proof sketch.* Suppose  $Q \mid R \approx x!(0)$  with  $Q, R \not\approx 0$ . Since  $x!(0)$  does not reduce and has exactly the barb  $x$ , neither  $Q$  nor  $R$  may reduce, at most one carries a barb, and the other is a stuck term. A stuck term is not barbed congruent to 0: if it is a receipt at some name  $z$ , the context  $\text{for}(w \leftarrow z)\omega!(0) \mid z!(0) \mid [\cdot]$  distinguishes it from 0, since supplying a datum at  $z$  lets the term consume it and the surrounding receipt then cannot. Composing with the divisibility statement: the multiplicity of the prime  $x!(0)$  in itself is 1, which no  $n \geq 2$  divides.  $\square$

The picture is a dichotomy, and neither half is where replication lives.

**Principle 3.7** (The two regimes). In the free regime, roots exist only by an accident of multiplicity: almost no process has one. In the idempotent regime, roots exist but say nothing: a process is its own root. A replicated system is in neither, because it takes an arbitrary  $P$  — with no divisibility to appeal to and no idempotence to fall back on — and produces something whose  $n$ -fold parallel composition acts like  $P$ . It can only do so by changing what parallel composition means. The coordinating process of the next sections is precisely that change: it is the price of embedding the process monoid in a divisible hull, and consensus is that price paid at runtime.

## 4 Roots by structural recursion

The sketch proposes computing a root from the syntax of a process. The clauses are these, with  $\sqrt{\cdot}$  defined simultaneously on processes and on names.

**Definition 4.1** (The homomorphic root).

$$\begin{aligned}
\sqrt{0} &= 0 & \sqrt{*x} &= *\sqrt{x} \\
\sqrt{\text{for}(y \leftarrow x)P} &= \text{for}(\sqrt{y} \leftarrow \sqrt{x})\sqrt{P} & \sqrt{x!(P)} &= \sqrt{x}!(\sqrt{P}) \\
\sqrt{@P} &= @\sqrt{P} & \sqrt{P \mid Q} &= \sqrt{P} \mid \sqrt{Q} \quad \text{if } P\#Q
\end{aligned}$$

where  $P\#Q$  asserts that  $P$  and  $Q$  do not interact.

**Lemma 4.2** (Injectivity).  $\sqrt{\cdot}$  is injective on processes and on names.

*Proof.* Each clause is headed by a distinct term constructor and is injective in its arguments; induct.  $\square$

**Lemma 4.3** (Substitution).  $\sqrt{P\{\overline{Q}/y\}} = \sqrt{P}\{\overline{Q}/\sqrt{y}\}$ .

*Proof.* By induction on  $P$ . The only clause requiring comment is the quote. Substitution in the rho calculus does not descend under  $@$ , while  $\sqrt{\cdot}$  does; but both sides then leave the quoted subterm's occurrences of  $\sqrt{y}$  untouched, since the left-hand side does not substitute there and the right-hand side does not either. Injectivity (Lemma 4.2) rules out a collision creating a substitution on the right that the left does not perform.  $\square$

Lemma 4.3 is what makes an inductive correctness argument possible at all, and it is also the reason the payload delivered by an arbiter must be  $\sqrt{Q}$  and not  $Q$ : the sketch's decision table writes the unrooted payload, which breaks the induction hypothesis at the first delivery.

## 4.1 Multiplicity is observable

**Proposition 4.4.**  $\sqrt{x!(0)} \mid \sqrt{x!(0)} \not\approx x!(0)$ .

*Proof.* Take  $C = \text{for}(y \leftarrow \sqrt{x})\text{for}(z \leftarrow \sqrt{x})\omega!(0) \mid [\cdot]$ . Then  $C[\sqrt{x}!(0) \mid \sqrt{x}!(0)] \Downarrow_\omega$  and the corresponding one-message term does not.  $\square$

No race occurs in Proposition 4.4; the failure is pure counting, and it is unavoidable for any map that sends a message to a message. It forces the correctness statement to be relative to a class of contexts, and it tells us which class: the observer must be replicated along with the observed, exactly as a client of a replicated service either is itself replicated or sits behind an adapter that hides the multiplicity. We therefore take the following as the statement to be proved.

**Definition 4.5** (Root of a system).  $R$  is a *square root* of  $P$  iff for every rho calculus context  $C$  and every name  $\omega$ ,

$$C[P] \Downarrow_\omega \iff (\sqrt{C})[R] \mid (\sqrt{C})[R] \Downarrow_{\sqrt{\omega}},$$

where  $\sqrt{C}$  is the image of  $C$  under Definition 4.1, extended to the hole by  $\sqrt{[\cdot]} = [\cdot]$ . The  $n$ -ary version replaces the two copies by  $n$ .

**Remark 4.6.** Definition 4.5 is  $\approx_{\mathcal{D}}$  for  $\mathcal{D}$  the image of the root map, and it is the same manoeuvre as the  $\mathcal{D}$ -relative full abstraction used elsewhere in this programme for the encoding of the  $\pi$ -calculus into the rho calculus: an absolute claim is unavailable, a claim relative to the image contexts is exactly what the construction supports, and the residues the construction leaves behind live on names outside that image.

## 4.2 The determinate fragment

**Definition 4.7** (Contention site, candidate, determinacy). A *contention site* in a state  $S$  is a name  $x$  together with the multiset of pending messages at  $x$  and the multiset of receipts at  $x$ . A *candidate* at the site is a matching of receipts to messages. The site is *determinate* iff all candidates lead to weakly barbed bisimilar successors.  $S$  is determinate iff every site in every state reachable from  $S$  is.

**Theorem 4.8** (Homomorphic root on the determinate fragment). *If  $P$  is determinate then  $\sqrt{P}$  of Definition 4.1 is a square root of  $P$  in the sense of Definition 4.5.*

*Proof sketch.* Doubling a determinate state can create contention where there was none — for  $(y \leftarrow x)Q \mid x!(R)$  has one candidate, its double has two — but every candidate of the doubled site is a pairing of copies of the same message with copies of the same receipt, so all candidates coincide up to  $\equiv$ , and the doubled site is determinate. Set

$$\mathcal{R} = \{ (\sqrt{S} \mid \sqrt{S'}, S'') : S, S' \text{ reachable from } P, S'' \text{ a common reduct} \},$$

and check that  $\mathcal{R}$  is a weak barbed bisimulation up to  $\equiv$ : a step of  $S''$  is matched by the corresponding step in each copy, taken in either order, and determinacy plus the diamond property reconvergence closes the square. Barbs correspond by Lemma 4.2, and payload correspondence is Lemma 4.3. Multiplicity is invisible because the observing context is doubled too.  $\square$

Theorem 4.8 is the positive content of the structural recursion, and it is worth stating that the recursion's side condition  $P\#Q$  is stronger than it needs to be: what is required is not absence of interaction but determinacy of the interaction.

## 5 The obstruction

**Proposition 5.1** (The race obstruction). *Let  $Q_1 = a!(0)$ ,  $Q_2 = b!(0)$ , and*

$$P = \text{for}(y \leftarrow x)*y \mid x!(Q_1) \mid x!(Q_2).$$

*Then  $\sqrt{P}$  of Definition 4.1 is not a square root of  $P$ .*

*Proof.*  $P$  has exactly two maximal reducts,  $Q_1 \mid x!(Q_2)$  and  $Q_2 \mid x!(Q_1)$ ; in each, exactly one of the barbs  $a, b$  is exposed, the other payload remaining sealed inside a message. Now

$$\begin{aligned} \sqrt{P} \mid \sqrt{P} &= \text{for}(\sqrt{y} \leftarrow \sqrt{x})*\sqrt{y} \mid \sqrt{x}!(\sqrt{Q_1}) \mid \sqrt{x}!(\sqrt{Q_2}) \\ &\quad \mid \text{for}(\sqrt{y} \leftarrow \sqrt{x})*\sqrt{y} \mid \sqrt{x}!(\sqrt{Q_1}) \mid \sqrt{x}!(\sqrt{Q_2}), \end{aligned}$$

in which two receipts face four messages. The candidate assigning  $\sqrt{Q_1}$  to the first receipt and  $\sqrt{Q_2}$  to the second yields a state exposing  $\sqrt{a}$  and  $\sqrt{b}$  simultaneously. Taking  $C = [\cdot]$  and  $\omega = b$  in Definition 4.5 together with the run of  $P$  that exposes  $a$ : the two sides disagree.  $\square$

The failure is not an artefact of the clauses. Whatever the map, the two copies each contain a decision to make, they make it independently, and nothing in the term relates the two decisions. The repair therefore cannot be a better recursion; it has to be a term that relates them.

## 6 The arbiter

Fix the site of Proposition 5.1: one receipt, two messages with distinct payloads. We split the delivered name  $\sqrt{x}$  into two *deposit* names  $(\sqrt{x})^\ell$  and  $(\sqrt{x})^r$ , one per message of the contention set, and let the root of the site be

$$\sqrt{\left[ \text{for}(y \leftarrow x) P \mid x!(Q_1) \mid x!(Q_2) \right]} = \text{for}(\sqrt{y} \leftarrow \sqrt{x}) \sqrt{P} \mid (\sqrt{x})^\ell!(\sqrt{Q_1}) \\ \mid (\sqrt{x})^r!(\sqrt{Q_2}) \mid C,$$

where  $C$  is the arbiter of Definition 6.2, the coordination channel is

$$o = @(*(\sqrt{x})^\ell \mid *(\sqrt{x})^r),$$

and *park* is a parking name derived in the same way.

**Remark 6.1** (On the derived names). The tag structure  $\{\ell, r\}$  is the address extension of Abramsky’s contraction and copy rules [1], and it is the same device used elsewhere in this programme to give the rho calculus a namespace discipline without a name server: a name is split, and the split is later rejoined by quoting a composite of its parts. The coordination name  $o$  is derived rather than fresh, so it is unguessable but not unforgeable — any context that knows the two deposit names can construct it. That is tolerable here because the correctness statement is relative to image contexts, and it is exactly the distinction that a serious treatment of namespace pragmatics has to price.

**Definition 6.2** (The arbiter).

$$C = \text{for}(q_1 \leftarrow (\sqrt{x})^\ell \ \& \ q_2 \leftarrow (\sqrt{x})^r) \{ \\ o!(\text{coin}) \\ \mid \text{for}(e_1 \leftarrow o \ \& \ e_2 \leftarrow o) \{ \\ \text{match } (*e_1, *e_2) \text{ with } \{ \\ (0,0) \Rightarrow \sqrt{x}!(*q_1) \mid \text{park}!(*q_2) \mid o!(0) \mid o!(0) \\ (1,1) \Rightarrow \sqrt{x}!(*q_1) \mid \text{park}!(*q_2) \mid o!(1) \mid o!(1) \\ (0,1) \Rightarrow \sqrt{x}!(*q_2) \mid \text{park}!(*q_1) \mid o!(0) \mid o!(1) \\ (1,0) \Rightarrow \sqrt{x}!(*q_2) \mid \text{park}!(*q_1) \mid o!(1) \mid o!(0) \\ \} \\ \} \\ \}$$

where *coin* abbreviates a local two-message race, new  $c$  in  $\{c!(0) \mid c!(1) \mid \text{for}(b \leftarrow c) \dots\}$ , whose outcome is whatever the copy’s own interpreter chooses.

Three lemmas carry the construction. Throughout, “the doubled term” means  $\sqrt{S} \mid \sqrt{S}$  where  $\sqrt{S}$  is the rooted site above, so that there are two arbiters, two messages under each tag, and two receipts at  $\sqrt{x}$ .

**Lemma 6.3** (Progress). *In the doubled term, the two arbiters between them consume all four deposits, and each acquires one message under each tag. If the multi-binding receipts of Definition 6.2 are read as sequential groups rather than as joins, there is a reachable deadlock.*

*Proof.* The outer receipt is an atomic two-way join, so an arbiter either takes one datum from each tag or takes nothing; with two data under each tag the two arbiters are served exactly once each. Read sequentially, the schedule in which the first arbiter takes both  $\ell$ -deposits and the second takes both  $r$ -deposits leaves both blocked on their second binding.  $\square$

**Lemma 6.4** (Exhaustion). *At every point of the arbitration, the channel  $o$  carries exactly two data, and the inner join is therefore taken by at most one arbiter at a time; the arbiter that takes it restores the pair it consumed. Consequently both arbiters read the same multiset of contributions.*

*Proof.* Each arbiter contributes one datum, so the multiset has size two once both have contributed. The inner join consumes both atomically and each branch of the table re-emits both. The other arbiter's inner join therefore sees the same two values, possibly assigned to its patterns in the other order.  $\square$

**Lemma 6.5** (Agreement). *Let the decision be  $f(e_1, e_2)$  for a function  $f$  into  $\{\text{left}, \text{right}\}$ . Both arbiters decide alike for every schedule iff  $f$  is symmetric,  $f(u, v) = f(v, u)$ . The table of Definition 6.2 is symmetric; it is exclusive-or.*

*Proof.* By Lemma 6.4 the two arbiters see the same pair of values and differ only in the order of assignment to  $e_1, e_2$ ; nothing in the term fixes that order. Symmetry is therefore necessary and sufficient.  $\square$

**Lemma 6.6** (The shared coin). *With  $f$  exclusive-or, the decision is the parity of the two contributions. If either copy's local race is fair, the decision is unbiased, and no single copy can determine it alone.*

Lemma 6.6 is the classical shared-coin argument, and it says why the sketch's table is the right one rather than merely a symmetric one: symmetry buys agreement, and parity buys the property that agreement is not agreement on one copy's preference.

**Theorem 6.7** (Single-site root). *Let  $P$  be the process of Proposition 5.1 and  $\sqrt{P}$  the rooted site above, with the delivered payloads rooted. Then  $\sqrt{P}$  is a square root of  $P$  in the sense of Definition 4.5.*

*Proof sketch.* By Lemma 6.3 both arbiters fire; by Lemmas 6.4 and 6.5 they deliver the same payload  $\sqrt{Q_i}$ ; so the doubled term reaches  $\sqrt{x}!(\sqrt{Q_i}) \mid \sqrt{x}!(\sqrt{Q_i})$  facing two receipts, a determinate site, and Theorem 4.8 applies from there, with Lemma 4.3 supplying the payload correspondence. The state so reached is the double of  $\sqrt{\cdot}$  applied to the reduct of  $P$  that selects  $Q_i$ , except for the parked loser and the residue on  $o$ , both of which live on names outside the image of the root map and are therefore invisible to the contexts of Definition 4.5. Since both values of  $i$  are reachable, the correspondence holds in both directions.  $\square$

**Remark 6.8** (The loser must be parked, and not returned). The reduct of  $P$  retains the losing message and its barb at  $x$ , so the arbiter cannot simply discard the loser; the sketch does discard it. It also cannot return it to  $\sqrt{x}$ : two arbiters would return two copies of the loser alongside two copies of the winner, four messages facing two receipts, and the receipts could then split — which is Proposition 5.1 again, one round later. The loser is therefore parked on a derived name, whence a later arbitration may retrieve it. The situation is worth naming: *arbitration does not consume a race, it defers the rest of it*, which is the reason the next section's arbiter has to persist.



**Remark 6.9** (Residue). Two contributions remain on  $o$  forever. They are unreachable by image contexts, but they are not nothing: in a cost-accounted reading they are storage held open by an arbitration that has completed, which is exactly the occupancy cost that a treatment of rent must charge and that a treatment of reduction cost does not.

## 7 From one race to every race

Definition 6.2 arbitrates a race that the writer of the root map could see. That is the limitation the rest of the construction has to remove, and the argument is short.

**Principle 7.1** (Contention is dynamic). Whether a name is contended is a property of reachable states, not of syntax. A message and a receipt that contend may be created by different branches of the term, brought together by substitution, or manufactured by an environment. A root map defined by structural recursion cannot locate the contention sites, so it cannot install arbiters at them.

The consequence is that arbitration must be attached to names rather than to races. The uniform version of the construction replaces every send with a deposit, every receipt with a delivery request, and installs, per name, an arbiter owning the traffic:

$$\begin{aligned}\sqrt{x!}(P) &= (\sqrt{x})^{\text{dep}!}(\sqrt{P}), \\ \sqrt{\text{for}(y \leftarrow x)P} &= (\sqrt{x})^{\text{req}!}(\dots) \mid \text{for}(\sqrt{y} \leftarrow (\sqrt{x})^{\text{del}})\sqrt{P}, \\ \sqrt{\cdot} &\text{ installs } A_{\sqrt{x}} \text{ for each name } \sqrt{x}.\end{aligned}$$

Each copy runs its own  $A_{\sqrt{x}}$ ; the copies of  $A_{\sqrt{x}}$  agree, by the mechanism of Section 6, on which deposit is delivered to which request, and then deliver it. Two things change relative to the single-site case, and both are the substance of the general problem rather than bookkeeping.

First, the general contention site has  $m$  requests and  $k$  deposits, so the arbiters must agree not on a bit but on a *matching* — a candidate in the sense of Definition 4.7, which for the store of an implementation is the zip of a bag of produces against a bag of consumes. Agreement on a bit generalises to agreement on a permutation, and a symmetric decision function generalises to a function of the multiset of contributions valued in matchings. The shared coin still works: the parity of the contributions selects an index into an enumeration of the candidates that both copies compute identically from the same multiset.

Second, once the arbiter owns the name it does not terminate. It agrees on one rendezvous, delivers, retrieves the parked remainder, and agrees again. What it produces over its lifetime is an agreed serialisation of all traffic at that name.

**Principle 7.2** (The fixed point of the construction). Applying the root map uniformly does not yield a process with arbiters sprinkled through it. It yields, per namespace, a replicated agent that accepts requests and deposits, agrees with its siblings on an order, and executes in that order. That agent is a node. The root of a process is what a node runs; agreement on the serialisation is what makes  $n$  of them compose to one.

We do not develop the uniform construction here. What Section 6 establishes is that the mechanism exists and what its correctness conditions are; what this section establishes is that a static placement of that mechanism cannot be complete, and where the complete version must live.

## 8 Resolvers

The rho calculus says which reductions are possible and says nothing about which one happens. That silence is usually harmless: an implementation supplies the missing datum — in a conventional interpreter, the physical races of cores against a store — and the language is indifferent to how. It is convenient to name the missing datum.

**Definition 8.1** (Resolver). A *resolver* is a map assigning, to each state and each contention site in it, a choice of candidate (or a distribution over candidates). The meaning of a program is a function of the pair (term, resolver), not of the term alone.

This reading is the one taken in the graded where-clause line of work, where the value algebra of a guard is precisely a specification of the resolver’s policy, and where quantum, probabilistic and Boolean readings of a program are the same term under three resolvers. The root equation puts a different demand on the same object.

**Principle 8.2** (The root problem is a resolver problem).  $R \parallel^n \approx P$  asks that  $n$  copies, each resolved by its own resolver, be indistinguishable from one copy resolved by one resolver. Call a family  $(q_1, \dots, q_n)$  *coherent for  $P$*  when, at every contention site of the  $n$ -fold composite, the choices the family makes are the image of a single choice at the corresponding site of  $P$ . Then: the homomorphic root of Definition 4.1 is a root exactly when the family is coherent, Theorem 4.8 is the observation that determinacy makes every family coherent for free, and the arbiter is a term that manufactures coherence from an incoherent family, at the price of one round of communication per site.

Two consequences are worth recording.

**Remark 8.3** (Why no recursion could have worked). The rho calculus is silent about resolution, so no term can *describe* a resolver. But a term can *constrain* one: the arbiter does not tell the interpreter which candidate to pick, it removes the candidates it does not want by consuming their inputs and re-presenting a site with a unique outcome. The root equation is the first equation in this programme where the calculus’s silence about nondeterminism is not benign — the equation is simply unsatisfiable by a homomorphism — and the response is the same one taken elsewhere: move what was a parameter of the interpretation into the syntax. The graded where-clause moves the resolver’s *policy*; the arbiter moves the resolver’s *agreement*. They are two halves of one manoeuvre, and they compose: a graded arbiter is one whose shared coin is non-uniform, which is to say that weighted leader election is a graded shared coin.

**Remark 8.4** (The coin is a race). In the accompanying implementation the local coin is written as a two-message race with one receipt, and the value obtained is whatever the copy’s interpreter chooses. This is not a stopgap for a missing primitive. It is the point: the copy’s own resolver is the only source of nondeterminism available to it, and arbitration is the discipline of combining  $n$  such sources symmetrically so that the composite behaves as one.

## 9 What the calculus assumes about delivery

The caveat the sketch attaches to its arbiter — that messages are neither lost nor indefinitely delayed — bundles two assumptions that fail in different places, and the word *asynchrony* is routinely made to carry a third that is not an assumption at all. It is worth separating them before drawing any conclusion about consensus.

- (a) **Asynchronous sending.**  $x!(Q)$  has no continuation: a sender does not wait for a receiver. This is a presentational choice about the term language, made here and in the asynchronous  $\pi$ -calculus [4, 3], and it says nothing whatever about a network.
- (b) **Unbounded delay.** The reduction relation imposes no schedule and no fairness condition, so a datum may remain unmatched for arbitrarily many steps while other reductions proceed. This is the distributed-systems sense of asynchrony, and the rho calculus already has it: it is a property of the *resolver* of Section 8, which is free to keep choosing elsewhere, and not of the message discipline.
- (c) **Reliable delivery.** A datum, once emitted, is delivered exactly once, to some receipt at its name, uncorrupted. This the rho calculus does assume, and it is the assumption that a network violates. Note that ordering is not among the assumptions: data at a name form a multiset, so reordering is native and costs nothing to model.

**Remark 9.1.** Section 10 as first drafted, and the sketch before it, treated “lost” and “indefinitely delayed” as one caveat and called its removal asynchrony. They are not the same, they are not removed by the same move, and the calculus’s position on them is not the same: (b) is already granted, (c) is already assumed. What follows separates them by making (c) a term rather than a side condition.

## 9.1 Channels as processes

To speak about delivery one moves from a name to a *channel*: a process interposed between two names, carrying the traffic from one to the other. Where the calculus writes  $\text{for}(y \leftarrow x)P \mid x!(Q)$ , we write

$$\text{for}(y \leftarrow x_1)P \mid \text{Chan}(x_1, x_2) \mid x_2!(Q), \quad \text{Chan}(x_1, x_2) = \text{for}(y \leftarrow x_2) x_1!(*y).$$

**Proposition 9.2** (Transparency). *The honest forwarder is transparent: for every  $P$  and  $Q$ , the interposed term above is weakly barbed bisimilar to  $\text{for}(y \leftarrow x)P \mid x!(Q)$  up to the renaming of the interface, and the interposition costs one additional reduction per message.*

This is the standard forwarder or link result for asynchronous name-passing calculi [4, 5], and the qualification “up to the renaming of the interface” is the usual one: the composite has barbs at  $x_1$  and  $x_2$  where the original has one at  $x$ , which is again a statement relative to a class of observers rather than an absolute one.

What the reification buys is that Chan need not be honest.

**Definition 9.3** (Sinister channels). Writing coin for the local race of Definition 6.2:

$$\begin{aligned} \text{Drop}(x_1, x_2) &= \text{for}(y \leftarrow x_2) \text{ if coin then } x_1!(*y) \text{ else } 0 \\ \text{Dup}(x_1, x_2) &= \text{for}(y \leftarrow x_2) (x_1!(*y) \mid x_1!(*y)) \\ \text{Corrupt}(x_1, x_2) &= \text{for}(y \leftarrow x_2) x_1!(F(*y)) \quad \text{for some } F \\ \text{Delay}(x_1, x_2) &= \text{for}(y \leftarrow x_2) \text{ park}!(*y) \mid \text{for}(z \leftarrow \text{park}) x_1!(*z) \end{aligned}$$

Delay is worth a word: it does not delay in any timing sense, because the calculus has no clock. It converts a datum into a datum at another name, which the resolver may then decline to choose for as long as it likes. That is the whole content of (b), and it confirms that unbounded delay needs no channel at all — an honest medium already permits it. Loss, duplication and corruption are what genuinely need a sinister medium, and they are exactly what (c) rules out.

**Proposition 9.4** (The medium replicates for free). *Chan is a persistent receipt whose continuation does not contend, so by Proposition 3.5 it is idempotent and is its own  $n$ -th root for every  $n$ .*

Proposition 9.4 localises the difficulty of this note rather precisely. The transport layer needs no arbitration: replicating it is free, and the copies of the medium may be composed without coordination. Everything the arbiter is for lives in the *linear* traffic that the medium carries.

**Remark 9.5** (The fault model becomes a perturbation of a term). Section 10 says that making the consensus correspondence a theorem needs a fault model, and that no fault model is given here. With the medium reified, the shape such a model would take is visible: it is the class of systems obtained from  $\sqrt{S}$  by replacing the medium in some copies with a variant from Definition 9.3. Omission faults are Drop; a network-level Byzantine fault is Corrupt; a node-level Byzantine fault is a copy whose *arbiter*, rather than whose medium, departs from Definition 6.2. The point is that all of these are terms, so the fault model can be stated inside the calculus and the correctness of an arbiter against it becomes an ordinary equivalence question rather than a metalevel stipulation.

**Principle 9.6** (The arbiter is a channel). Definition 6.2 consumes data at the deposit names and emits at the delivered name. That is the shape of Chan, with a policy inside: the arbiter drops nothing but defers some of what it receives (Remark 6.8), and it chooses which. So the arbiter and the sinister channel are the same construction, and what distinguishes a consensus protocol from a faulty network is not the shape of the term but a guarantee *across* copies. A consensus protocol is a family of media, one per copy, constrained to make the same choices.

Principle 9.6 is Section 8’s slogan at the level of the medium, and it makes the internalisation pattern of this programme visible three times over in one construction: a graded where-clause moves the resolver’s policy into the syntax, the arbiter moves the resolver’s agreement into the syntax, and Chan moves delivery into the syntax. In each case a parameter of the interpretation becomes a term, and in each case what was an assumption one states about a model becomes a process one may reason about.

**Remark 9.7** (What existing protocols deliver). Assumption (c) is not extravagant. Over a live connection, TCP [6] delivers exactly once, uncorrupted, and in order — more than the calculus asks on ordering and precisely what it asks on reliability. What TCP does not supply is that the connection remains live; a partition is the failure of (b) and (c) together and is not detectable from inside. So the gap between the rho calculus’s message discipline and a real transport is smaller than the phrase “the network is unreliable” suggests, and the reified medium is there for the residue: partition, and adversarial peers.

## 10 The consensus analogy

The sketch’s second slide observes that a consensus protocol runs  $n$  copies of a node coordinating  $n$  copies of a contract, so arranged that the copies result in a single execution — which is the root equation. We present the correspondence as an analogy, and mark plainly what would be needed to make it a theorem.

| Consensus                                              | Arbitration, as above                                                                         |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Agreement: no two correct processes decide differently | Lemma 6.5: symmetry of the decision function                                                  |
| Validity: the decision is some proposal                | The decision selects a candidate of the site, so the delivered payload is one of the deposits |
| Termination                                            | Lemma 6.3: atomicity of the joins, under reliable delivery                                    |
| Randomised agreement                                   | Lemma 6.6: parity of the local coins                                                          |
| Ordering the log, not one value                        | Remark 6.8 and Section 7: the arbiter defers the remainder of the race and must persist       |
| Reliable channels                                      | Assumption (c) of Section 9, reified as Chan and removable term-by-term                       |

The sketch’s own caveat names the right two things, and Section 9 says where each of them lives. The arbiter assumes reliable delivery — assumption (c), which is the calculus’s own and which a sinister medium removes — and it assumes that the arbiter does not lie, which no medium can repair. It does *not* assume anything about delay: assumption (b) is already absent, since the resolver is under no fairness obligation, and Lemmas 6.3 and 6.4 are stated as reachability rather than as timing. So the two axes are unreliable delivery and dishonest participants, not “asynchrony” and Byzantine behaviour; and a protocol such as Casper exists to address both, together with the delay the calculus already permits, in a decentralised setting.

Two remarks about the shape a theorem would take, offered as orientation and not as results. First, the correspondence has an obvious direction: an arbiter that is correct in a model with crash faults would decide a binary value agreed by all copies, which is what the classical impossibility result for deterministic asynchronous consensus [9] forbids — so in such a model the coin of Lemma 6.6 would not be a stylistic choice but a necessity, and the neighbourhood to look in is randomised agreement [2]. The separation of Section 9 is what makes this remark say anything: the impossibility result assumes *reliable* channels and drops only the timing assumption — (c) held, (b) absent — which is exactly the position the rho calculus is already in, and which is therefore the setting the arbiter of Section 6 would have to be judged in first. Second, the general site of Section 7 asks for agreement on a serialisation rather than on a value, which is atomic broadcast, and hence the replicated state machine construction [14]. Making either remark precise requires a fault model, a definition of correctness for an arbiter under it, and a reduction; none of that is attempted here.

## 11 $n$ -th roots

The sketch closes by setting the  $n$ -ary case as an exercise. It goes through, with the following changes.

- The deposit tags range over the contention set rather than over  $\{\ell, r\}$ , and the outer join has the arity of that set.
- Each of the  $n$  copies contributes one value on  $o$ , and the inner join has arity  $n$ . Each copy consumes all  $n$  contributions atomically and re-emits all  $n$ .

- The decision function must be symmetric in  $n$  arguments — a function of the multiset — for the reason of Lemma 6.5, since each copy assigns the same multiset to its patterns in its own order. Parity generalises to the sum modulo the number of candidates.

**Lemma 11.1** ( *$n$ -ary exhaustion*). *With exactly  $n$  contributions present and an atomic  $n$ -fold join, at most one copy is reading at any time, and it restores what it consumed; so all  $n$  copies read the same multiset.*

**Proposition 11.2** (*Cost of a site*). *Arbitrating one contention site among  $n$  copies costs  $n$  contributions,  $n$  inner rendezvous, and  $n^2$  re-emissions:  $\Theta(n)$  rendezvous and  $\Theta(n^2)$  messages, plus a residue of  $n$  messages held on  $o$ .*

The quadratic message count is the familiar shape of all-to-all agreement, and it is not an artefact of writing the protocol in this calculus: it is the cost of every copy learning every contribution. In a cost-accounted reading, this is a levy assessed per contention site, which suggests the design rule that one buys fault tolerance by the race and not by the program — a determinate program is replicated for free, by Theorem 4.8.

## 12 The implementation

Appendix A gives the square root of a racing rholang process, in two parts. The first is the naive homomorphic root of Definition 4.1, doubled, which reaches a state exposing both barbs and so exhibits Proposition 5.1 concretely. The second is the coordinated root: two copies, each carrying its own arbiter, its own coin written as a local race, and the symmetric decision table, delivering the agreed payload on the delivered channel and parking the loser.

The corrections of Section 6 are all visible in the listing: the multi-binding receipts are joins, written  $\&$ ; the delivered payloads are the rooted ones; the loser is parked rather than dropped or returned; and the residues sit on names outside the image of the root map. The file has not been executed against an interpreter.

## 13 Future work

**Threshold roots.** The construction of this note replicates faults rather than tolerating them: an exact root needs all  $n$  copies, and losing one loses  $P$ . The weakening that repairs this asks that *any*  $k$  of the  $n$  copies compose to  $P$ , which is a quorum condition rather than an equation, and which puts quorum intersection where divisibility was in Section 3. That is where the classical resilience bounds would enter, and it is the obvious sequel.

**Unique decomposition for the rho calculus.** Hypothesis 3.3 is assumed and not proved. Reflection and infinite behaviour both put the existing results out of reach, and either a proof or a counterexample would sharpen Proposition 3.4 from a picture into a theorem.

**The uniform arbiter.** Section 7 specifies the per-namespace arbiter and does not construct it. Constructing it, and proving the operational correspondence relative to image contexts, is the main piece of work this note leaves open.

**The arbiter against a sinister medium.** Remark 9.5 says where a fault model would live but does not build one. The first question to ask of it is narrow and answerable: which of Lemmas 6.3, 6.4 and 6.5 survive the replacement of one copy's medium by Drop, and which by Dup. Duplication looks the more dangerous of the two, since Lemma 6.4 counts.

**Cost and occupancy.** The residues of Section 6 are held storage, not performed reductions, and pricing them belongs to the same unfinished treatment of namespace pragmatics that owns the derived-name question.

**Graded arbitration.** Lemma 6.6 uses a uniform coin. A graded arbiter would weight the contributions, and the resulting non-uniform shared coin is a weighted leader election; stating the agreement lemma over a general value algebra is a small piece of work with an interesting reading.

## 14 Conclusion

The equation  $\sqrt{P} \mid \sqrt{P} \approx P$  looks like a curiosity about parallel composition and turns out to be a compact statement of what replication is for. Read arithmetically, it says that exact roots are governed by divisibility and hence are rare, so a system that manufactures them for arbitrary  $P$  must be changing what parallel composition means. Read operationally, it says that the obstruction lies exactly at contention, that no structural recursion can remove it, and that the removal is a term which forces independently resolved copies to resolve alike. Read as a design, it says that because contention is dynamic the removing term cannot be attached to a race but must own a namespace — and a replicated agent that owns a namespace and agrees with its siblings on the order of its traffic is a node.

The distance between the arbiter of Section 6 and a protocol that works on the open Internet is exactly the two assumptions the original sketch named: that messages arrive, and that the arbiter is honest. It is not, as one is tempted to say, a matter of asynchrony: the calculus imposes no schedule and no fairness, so unbounded delay is granted already, and what a network takes away is delivery. Once the medium is a process, that difference is a difference between terms, and the arbiter turns out to be a medium too — one that  $n$  copies are obliged to agree on. That the distance is only those two assumptions is the point of the exercise.

## Acknowledgement

Claude (Anthropic) assisted with the review of the source material, the drafting of this note, and the checking of its statements.

## A `sqrt.rho`

```
// =====  
// sqrt.rho -- the square root of a rholang process  
//  
// Companion artifact to "Radical Fault Tolerance".  
//  
// The source process is a two-message race:
```

```

//
// P = for( y <- x ) *y | x!( @ "a"!(0) ) | x!( @ "b"!(0) )
//
// P has exactly two maximal reductions. One exposes a barb on "a"
// and leaves a message on x carrying @ "b"!(0); the other exposes a
// barb on "b" and leaves a message on x carrying @ "a"!(0). In no
// reachable state of P are both "a" and "b" exposed.
//
// PART 1 shows the naive homomorphic root doubled with itself. It
// reaches a state exposing BOTH barbs, so it is not a root.
//
// PART 2 shows the coordinated root. Each copy carries an arbiter;
// the two arbiters agree, so exactly one of the two barbs is exposed
// (twice), and the loser is parked rather than lost.
//
// Notation is rholang 1.4: '<- ' is a linear receipt, '&' joins within
// a group, ';' separates groups. Every multi-binding receipt below is
// a JOIN and must be written with '&': with ';' the two arbiters can
// each take one half of what they need and deadlock.
//
// NOT EXECUTED: this file has not been run against an interpreter in
// the session that produced it.
// =====

// -----
// PART 1. The naive root, doubled. BROKEN.
//
// sqrt[ for( y <- x ) *y | x!(Q1) | x!(Q2) ]
// = for( sy <- sx ) *sy | sx!(sqrt Q1) | sx!(sqrt Q2)
//
// Doubling gives two receivers and four messages on sx. Nothing
// forces the two receivers to take payloads from the same side of
// the original race, so the term below can expose "a" and "b" at
// once -- a state with no counterpart in P.
// -----

new naive in {
  contract naive( @copy ) = {
    for( sy <- @ "sx" ) { *sy }
    | @ "sx"!( @ "a"!(0) )
    | @ "sx"!( @ "b"!(0) )
  }
  |
  naive!( 1 ) | naive!( 2 )
}

|

// -----
// PART 2. The coordinated root.
//
// The channel sx is split into two tagged deposit channels, xl and
// xr, one per message of the original contention set. Each copy
// deposits its own pair, so after doubling xl carries two messages
// and xr carries two messages.
//

```



```

// Each copy runs an arbiter.  An arbiter
// (1) joins on one message from each tag -- so the two arbiters
// partition the deposits between them, one pair each;
// (2) contributes one bit to the shared channel 'out';
// (3) joins on TWO bits from 'out', decides by a function that is
// SYMMETRIC in the pair, re-emits the pair it consumed so its
// sibling reads the same multiset, delivers the winner on sx,
// and parks the loser.
//
// Symmetry of the decision table is what makes the two arbiters
// agree: they may read the two bits in either order. The table is
// exclusive-or -- agree on Q1, disagree on Q2 -- which is also what
// makes the shared bit unbiased when either copy's local coin is.
//
// The local coin is a race, not a primitive: two messages, one
// receiver. The nondeterminism the interpreter uses to resolve that
// race IS the copy's resolver, exposed as a term. Arbitration is
// then the business of turning two independent resolvers into one.
// -----

new root, xl, xr, sx, park, out in {

  contract root( @copy ) = {

    // ---- the rooted receiver: the image of for( y <- x )*y
    for( sy <- sx ) { *sy }

    // ---- the rooted messages, deposited under their tags
    | xl!( @ "a"! (0) )
    | xr!( @ "b"! (0) )

    // ---- this copy's arbiter
    | for( q1 <- xl & q2 <- xr ) {
      new coin in {
        // the local resolver, as a race
        coin!( 0 ) | coin!( 1 )
        | for( c <- coin ) {
          // contribute this copy's bit
          out!( *c )
          // read the whole multiset of contributions
          | for( e1 <- out & e2 <- out ) {
            match ( *e1, *e2 ) {
              // agree => the left deposit wins
              ( 0, 0 ) => { sx!( *q1 ) | park!( *q2 ) | out!( 0 ) | out!( 0 ) }
              ( 1, 1 ) => { sx!( *q1 ) | park!( *q2 ) | out!( 1 ) | out!( 1 ) }
              // disagree => the right deposit wins
              ( 0, 1 ) => { sx!( *q2 ) | park!( *q1 ) | out!( 0 ) | out!( 1 ) }
              ( 1, 0 ) => { sx!( *q2 ) | park!( *q1 ) | out!( 1 ) | out!( 0 ) }
            }
          }
        }
      }
    }

    |
    root!( 1 ) | root!( 2 )
  }
}

```

```

// -----
// What to observe.
//
// PART 1 can reach a state with a barb on "a" and a barb on "b".
// PART 2 cannot: both arbiters compute the same function of the same
// pair of bits, so both deliver the same payload, and the doubled
// system exposes one barb twice. Two bits remain on 'out' and two
// messages remain on 'park'; both residues live on names outside the
// image of the root map, which is why the correctness statement is
// relative to the image contexts and not to arbitrary ones.
//
// Read as a protocol rather than as a term: 'out' is a shared coin,
// the symmetric table is the agreement rule, 'park' is the loser's
// place in the queue, and the join arities are the quorum. The
// n-copy version replaces the pair by an n-fold join and the table
// by any function symmetric in n arguments; each copy then re-emits
// the n contributions it consumed, so a contention site costs n
// rendezvous and  $O(n^2)$  messages.
// -----

// -----
// PART 3. The medium, reified.
//
// The calculus assumes exactly-once, uncorrupted delivery. It assumes
// nothing about delay: the reduction relation carries no schedule and
// no fairness condition, so a datum may sit unmatched indefinitely
// while other reductions proceed. Those are different assumptions and
// they fail in different places -- delay is the resolver's business,
// delivery is the medium's. To speak about delivery, interpose a
// process between two names.
//
//   for( y <- x )P | x!( Q )
//   becomes
//   for( y <- x1 )P | Chan( x1, x2 ) | x2!( Q )
//
// An honest Chan forwards and is transparent up to weak bisimulation,
// at the cost of one reduction per message. It is also a persistent
// receipt, hence idempotent, hence its own n-th root: the transport
// layer replicates for free, and everything the arbiter of PART 2 is
// for lives in the linear traffic the medium carries.
// -----

new Chan, Drop, Dup, Corrupt, Delay in {

  // honest: forward
  contract Chan( x1, x2 ) = {
    for( y <= *x2 ) { *x1!( *y ) }
  }

  // omission: forward or discard, decided by the local resolver
  | contract Drop( x1, x2 ) = {
    for( y <= *x2 ) {
      new coin in {
        coin!( 0 ) | coin!( 1 )
      }
      | for( b <- coin ) {

```

```

        match *b { 0 => { *x1!( *y ) } 1 => { Nil } }
    }
}

// duplication: note this is the variant that threatens Lemma
// "exhaustion" in the note, since that lemma counts
| contract Dup( x1, x2 ) = {
    for( y <= *x2 ) { *x1!( *y ) | *x1!( *y ) }
}

// corruption: a network-level Byzantine fault
| contract Corrupt( x1, x2 ) = {
    for( y <= *x2 ) { *x1!( @"corrupted"!( *y ) ) }
}

// "delay": no clock is involved. The datum is moved to another
// name, which the resolver may decline to choose for as long as it
// likes -- which is why unbounded delay needs no sinister medium.
| contract Delay( x1, x2 ) = {
    new held in {
        for( y <= *x2 ) { held!( *y ) }
        | for( z <= held ) { *x1!( *z ) }
    }
}

// -----
// The arbiter of PART 2 has the shape of a Chan: it consumes at the
// deposit names and emits at the delivered name, with a policy inside.
// The difference between a faulty network and a consensus protocol is
// not the shape of the term but a guarantee across copies -- that the
// n media make the same choices.
// -----

```

## References

- [1] S. Abramsky. Computational interpretations of linear logic. *Theoretical Computer Science* 111(1-2):3-57, 1993.
- [2] M. Ben-Or. Another advantage of free choice: completely asynchronous agreement protocols. *PODC*, 1983.
- [3] G. Boudol. Asynchrony and the  $\pi$ -calculus. INRIA Research Report 1702, 1992.
- [4] K. Honda and M. Tokoro. An object calculus for asynchronous communication. *ECOOP*, 1991.
- [5] M. Merro and D. Sangiorgi. On asynchrony in name-passing calculi. *Mathematical Structures in Computer Science* 14(5):715-767, 2004.
- [6] J. Postel (ed.). Transmission Control Protocol. RFC 793, 1981.
- [7] M. Blum. Coin flipping by telephone: a protocol for solving impossible problems. *ACM SIGACT News* 15(1):23-27, 1983.

- [8] M. Castro and B. Liskov. Practical Byzantine fault tolerance. *OSDI*, 1999.
- [9] M. Fischer, N. Lynch and M. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM* 32(2):374–382, 1985.
- [10] B. Luttik. Unique parallel decomposition in branching and weak bisimulation semantics. *Theoretical Computer Science* 612:29–44, 2016.
- [11] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science* 141(5):49–67, 2005.
- [12] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- [13] R. Milner and F. Moller. Unique decomposition of processes. *Theoretical Computer Science* 107(2):357–363, 1993.
- [14] F. B. Schneider. Implementing fault-tolerant services using the state machine approach: a tutorial. *ACM Computing Surveys* 22(4):299–319, 1990.
- [15] L. G. Meredith. Radical fault tolerance. RChain Research deck, 2018.